SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	**CSA1223**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO5**	**Assessment:**	**ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)**
Weightage:	**30%**	**Total Marks:**	**25**
CO5 Statement:	*Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.*		
Student Name:	**V ADARSH REDDY**	**Reg.No.:**	**192525412**
Department:	**B TECH AI ML**	**Date:**	**26-08-2026**

ANNEXURE A

Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.

5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

I V ADARSH REDDY (192525412) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

V ADARSH REDDY
Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning

Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

I/O Architecture, Interrupts and DMA — Pseudocode Questions and Answers

Q1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.

Pseudocode

```
BEGIN DMA_Transfer(device, mem_addr, length)
  // 1. CPU sets up the transfer (only touches control
registers)  device.SOURCE_REG  = device.data_port
device.DEST_REG  = mem_addr  device.LENGTH_REG  =
length  device.MODE_REG  = DMA_MODE_BLOCK
device.INT_ENABLE  = TRUE

  // 2. CPU issues start command via a single MMIO write, then is free
device.CONTROL_REG  = START_TRANSFER
  CPU.resume_other_work()      // CPU not polling or copying data

  // 3. DMA controller performs the transfer autonomously
  WHILE device.bytes_remaining > 0 DO
    DMA_Controller.request_bus()
    DMA_Controller.transfer_block(device -> memory)
    DMA_Controller.release_bus()  // cycle-steal / burst per policy
END WHILE

  // 4. On completion, controller raises an interrupt (no CPU polling
needed)  device.STATUS_REG = TRANSFER_COMPLETE
RAISE_INTERRUPT(vector = device.IRQ_VECTOR) END

// Interrupt Service Routine — runs only once, at the
very end ISR DMA_Complete_Handler(vector)  device
= LOOKUP_DEVICE(vector)
  IF device.STATUS_REG == TRANSFER_COMPLETE THEN
NOTIFY_WAITING_PROCESS(device.owner_process)
    device.STATUS_REG = IDLE
    CLEAR_INTERRUPT(vector)
```

```
  ELSE IF device.STATUS_REG == TRANSFER_ERROR THEN
    HANDLE_DMA_ERROR(device)
  END IF
END ISR
```

The CPU only performs three MMIO register writes to start the transfer, then does other work; the DMA controller autonomously moves all data and interrupts the CPU exactly once at completion, minimizing CPU involvement to setup and teardown.

Q2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.

Pseudocode

```
STRUCT InterruptEntry
vector_id   : INTEGER
  priority    : INTEGER     // lower number = higher priority
isr_pointer : FUNCTION
```

```
END STRUCT

VECTOR_TABLE : ARRAY[0..MAX_VECTORS] OF InterruptEntry
PENDING_QUEUE : PRIORITY_QUEUE OF InterruptEntry   // ordered by
priority

// Called by hardware whenever any device asserts its
interrupt line ON_INTERRUPT_SIGNAL(vector_id)     entry =
VECTOR_TABLE[vector_id]
   PENDING_QUEUE.insert(entry, key = entry.priority)
   IF CPU.interrupts_enabled AND NOT CPU.currently_servicing THEN
DISPATCH_NEXT_INTERRUPT()
  END IF
END

PROCEDURE
DISPATCH_NEXT_INTERRUPT()
WHILE NOT
PENDING_QUEUE.is_empty() DO
     entry = PENDING_QUEUE.extract_min()  // highest priority first       CPU.
currently_servicing = TRUE

     SAVE_CONTEXT(CPU.registers)
     DISABLE_LOWER_OR_EQUAL_PRIORITY_INTERRUPTS(entry.priority)

     entry.isr_pointer()        // direct jump: vectored dispatch

     RESTORE_CONTEXT(CPU.registers)
     ENABLE_ALL_INTERRUPTS()
     CPU.currently_servicing = FALSE
  END WHILE
END

// Example device-specific ISRs registered into the vector table at boot
REGISTER_ISR(vector = 0x20, priority = 1, isr = NVMe_ISR)     // highest
```

```
REGISTER_ISR(vector = 0x21, priority = 2, isr = NIC_ISR)
REGISTER_ISR(vector = 0x22, priority = 3, isr = USB_ISR)      // lowest
```

Each device is pre-assigned a unique vector that maps directly to its ISR (true vectored dispatch, no polling). A priority queue orders simultaneous interrupts so higher-priority devices (e.g., storage) preempt lower-priority ones (e.g., USB), and lower/equal priority interrupts are masked during servicing to prevent priority inversion.

Q3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.

Pseudocode

```
ENUM AccessMode { MEMORY_MAPPED, IO_MAPPED }
```

```
FUNCTION SELECT_IO_MODE(device) RETURNS AccessMode    latency_req   =
device.required_latency_ns    bandwidth_req  = device.required_bandwidth_MBps

  IF bandwidth_req > BANDWIDTH_THRESHOLD THEN
     // High bandwidth: MMIO allows large block transfers and DMA
     RETURN MEMORY_MAPPED
  ELSE IF latency_req < LATENCY_THRESHOLD THEN
     // Very latency-sensitive, small transfers: dedicated IO space
     // avoids contention with cached memory traffic
     RETURN IO_MAPPED
ELSE
     // Default: prefer MMIO for uniform addressing and DMA
compatibility       RETURN MEMORY_MAPPED
  END IF
END

PROCEDURE ACCESS_DEVICE(device,
operation, data)    mode =
SELECT_IO_MODE(device)

  IF mode == MEMORY_MAPPED THEN
     addr = device.mmio_base + operation.register_offset
     IF operation == READ THEN
        RETURN MEM_READ(addr)       // ordinary load instruction
ELSE
        MEM_WRITE(addr, data)       // ordinary store instruction
END IF
  ELSE  // IO_MAPPED
     port = device.io_port_base + operation.register_offset
     IF operation == READ THEN
        RETURN IN(port)            // dedicated IN instruction
ELSE
        OUT(port, data)            // dedicated OUT instruction
END IF
  END IF
```

```
END
```

The selector checks the device's declared bandwidth and latency needs: high-bandwidth devices are routed to MMIO (enabling large transfers, caching, and DMA), while ultra-latency-sensitive, low-bandwidth devices can use dedicated I/O-mapped ports to avoid competing with memory bus traffic, keeping their access path short and predictable.

Q4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.

Pseudocode

```
STRUCT BusRequest    device_id   :
INTEGER    bus_type    : ENUM { PCIE, USB,
THUNDERBOLT }    priority    : INTEGER
   length     : INTEGER   // bytes requested
END STRUCT

REQUEST_QUEUE : LIST OF BusRequest
BUS_BUSY     : BOOLEAN = FALSE

PROCEDURE REQUEST_BUS(device_id, bus_type, priority,
length)    req = NEW BusRequest(device_id, bus_type,
priority, length)    REQUEST_QUEUE.append(req)
   IF NOT BUS_BUSY THEN
      ARBITRATE_AND_GRANT()
   END IF
END

PROCEDURE ARBITRATE_AND_GRANT()
   IF REQUEST_QUEUE.is_empty() THEN RETURN END IF

   // Rank requests: bus-type weight first, then priority, then FIFO order
sorted = REQUEST_QUEUE.sort_by(
            key = (BUS_WEIGHT[req.bus_type], req.priority, req.arrival_time))

   winner = sorted.first()
REQUEST_QUEUE.remove(winner)
   BUS_BUSY = TRUE

   GRANT_BUS(winner.device_id)
   TRANSFER_LENGTH = MIN(winner.length,
MAX_BURST_SIZE[winner.bus_type])
PERFORM_TRANSFER(winner.device_id, TRANSFER_LENGTH)

   BUS_BUSY = FALSE
```

```
   IF winner.length > TRANSFER_LENGTH THEN
winner.length -= TRANSFER_LENGTH
      REQUEST_QUEUE.append(winner)   // re-queue
remainder    END IF

   IF NOT REQUEST_QUEUE.is_empty() THEN
      ARBITRATE_AND_GRANT()        // continue servicing
queue    END IF
END

// Example static weighting (lower = serviced first)
BUS_WEIGHT[PCIE]        = 1   // highest priority: internal, latency-critical
BUS_WEIGHT[THUNDERBOLT] = 2   // tunnels PCIe traffic externally
BUS_WEIGHT[USB]         = 3   // best-effort peripherals
```

Requests from all three interfaces are funneled into a single arbitration queue and ranked by bus-type weight, device priority, and arrival order (fairness). The arbiter grants the bus for a bounded burst so no single device can monopolize it, re-queuing any remaining data — a scheme resembling weighted round-robin arbitration used in real PCIe switch fabrics.

Q5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Pseudocode

```
CONST SPEED_THRESHOLD = 10   // MBps; devices below this are polled


PROCEDURE HANDLE_DEVICE_IO(device)
```

```
  IF device.data_rate < SPEED_THRESHOLD THEN
    POLL_TRANSFER(device)
  ELSE
    DMA_INTERRUPT_TRANSFER(device)
  END IF
END


// --- Low-speed path: simple polling loop ---
PROCEDURE POLL_TRANSFER(device)
  WHILE device.has_more_data() DO
    WHILE device.STATUS_REG != READY DO
      // busy-wait; acceptable since device is
slow/infrequent      END WHILE      byte =
device.DATA_REG
    BUFFER.append(byte)
  END WHILE
END

// --- High-speed path: DMA offload with interrupt-driven completion ---
PROCEDURE DMA_INTERRUPT_TRANSFER(device)
  device.DMA_DEST  =
ALLOCATE_BUFFER(device.expected_length)
device.DMA_LENGTH = device.expected_length
device.INT_ENABLE = TRUE   device.CONTROL_REG =
START_DMA
  // CPU does NOT wait; it returns to the scheduler immediately
RETURN
END

ISR
High_Speed_Complete_Handler(vector)
device = LOOKUP_DEVICE(vector)
PROCESS_BUFFER(device.DMA_DEST)
device.STATUS_REG = IDLE
  CLEAR_INTERRUPT(vector)
```

```
END ISR

// --- Scheduler dispatch loop ---
MAIN_LOOP()
   FOR EACH device IN ACTIVE_DEVICES DO
      HANDLE_DEVICE_IO(device)
   END FOR
END
```

A single dispatcher routes each device by its data rate: slow devices (e.g., keyboard, sensors) are serviced with simple polling since the overhead of interrupts/DMA setup would exceed the benefit, while high-speed devices (e.g., NVMe, NIC) are handed to DMA engines with interrupt-driven completion so the CPU is never blocked waiting on them. This hybrid approach minimizes both wasted CPU cycles (from polling fast devices) and unnecessary interrupt overhead (from interrupting on every byte of a slow device).